Onyx — Changes applied to Circle

Onyx · multi-process OS on Raspberry Pi 4

Onyx uses **Circle** (rsta2/circle) as its hardware-abstraction + driver layer, kept as close to upstream as possible. The handful of changes we need live in a **fork**, pulled in as a **git submodule**:

- **Submodule:** `circle/` → `https://github.com/stephaneweg/circle.git` (`.gitmodules`), branch `onyx`.
- **Upstream base:** `https://github.com/rsta2/circle.git`, tag `Step51` (remote `upstream`).
- The superproject pins a specific fork commit; `git submodule status` shows it (currently `Step51-6-g…`).

The patches on top of `Step51` are small and surgical. The diffs below are the exact output of `git diff Step51..onyx` inside `circle/`. To reproduce:

```
git -C circle remote -v                  # origin = the fork, upstream = rsta2/circle
git -C circle log  --oneline Step51..onyx
git -C circle diff Step51..onyx
```

> **Design rule.** Patch Circle as little as possible. When a change can be header-only (no library rebuild), it is. Anything that can live in our own kernel instead (for example the cooked-mouse changed-button handling, done in `kernel/`'s mouse stub) is **not** a Circle patch.
>
> Some in-source comments still say "Zircon" (the project's former codename) instead of "Onyx" — cosmetic only.

Summary

#	Patch	Files touched	Rebuild
1	Keyboard map **decoupled** from the kernel (no compiled-in country maps; loaded at runtime) · `MAX_TASKS` 20 → 40	`input/keymap.{h,cpp}` (+ 7 `keymap_*.h` deleted), `usb/usbkeyboard.h`, `input/keyboardbehaviour.h`, `sysconfig.h`	`libinput` (+ kernel for `MAX_TASKS`)
2	Heap **large-block reuse** (fix per-launch canvas leak) + free-list byte accounting	`heapallocator.{h,cpp}`, `sysconfig.h`	`libcircle`
3	**Free-space accounting** in the page allocator + memory accessors (for the `meminfo` kapi)	`memory.h`, `pageallocator.{h,cpp}`	`libcircle` (`memory.h` header-only)

---

1. Keyboard map decoupled from the kernel (+ `MAX_TASKS`)

Why. Circle's cooked-mode keyboard bakes the country maps into the binary (one selected at boot via `keymap=`) and offers no way to change it afterwards. Onyx wants **live** layout switching *and* layouts that ship as **data files** (`SD:/etc/keymaps/*.kmap`), so a new layout needs no kernel rebuild — and the kernel carries **no** keyboard tables at all.

What.

1. **Expose the keyboard's `CKeyMap`** up the device stack (`keyboardbehaviour.h`, `usbkeyboard.h` gain a `GetKeyMap()`), so the kernel can reach the live map and fill it through the existing public `CKeyMap::SetEntry` / `ClearTable`.
2. **Drop the compiled-in maps.** `keymap.cpp` no longer `#include`s the `keymap_*.h` tables, and `CKeyMap` loses `s_DefaultMap[]`, `s_MapDirectory[]` and `LookupDefaultMap()`; the constructor just zeroes the map (every entry `KeyNone`). The 7 stock tables (`keymap_{de,dv,es,fr,it,uk,us}.h`, **905 lines**) are **deleted** from the Circle tree — they now live in the Onyx repo at `tools/keymaps/maps/` and are compiled to `.kmap` blobs by `tools/keymaps/genkeymaps.py`.

The kernel fills the live keyboard from a `.kmap` file via `SetEntry` (`kapi_set_keymap_data`, ABI v27), re-applying it whenever a keyboard (re-)attaches. `MAX_TASKS` goes **20 → 40**: the network stack adds long-lived background tasks (net / DHCP / WPA supplicant + NTP + IRC).

```
--- a/include/circle/input/keymap.h
+++ b/include/circle/input/keymap.h
@@ -130,8 +130,9 @@ public:

    u8 GetLEDStatus (void) const;

-private:
-   static const void *LookupDefaultMap (const char *pLocale);
+   // Zircon: the keyboard map is empty until the kernel fills it at runtime via
+   // ClearTable/SetEntry from a SD:/etc/keymaps/*.kmap file -- no country map is
+   // compiled in (see CKeyMap::CKeyMap and kapi_set_keymap_data).

 private:
    u16 m_KeyMap[PHY_MAX_CODE+1][K_CTRLTAB+1];
@@ -141,8 +142,6 @@ private:
    boolean m_bScrollLock;

    static const char *s_KeyStrings[KeyMaxCode-KeySpace];
-   static const u16 s_DefaultMap[][PHY_MAX_CODE+1][K_CTRLTAB+1];
-   static const char *s_MapDirectory[];
 };

--- a/lib/input/keymap.cpp
+++ b/lib/input/keymap.cpp
@@ -96,60 +96,16 @@ const char *CKeyMap::s_KeyStrings[KeyMaxCode-KeySpace] =
    "."          // KeyKP_Period
 };

-#define C(chr)     ((u16) (u8) (chr))
-
-const u16 CKeyMap::s_DefaultMap[][PHY_MAX_CODE+1][K_CTRLTAB+1] =
-{
-   {
-       #include "keymap_de.h"
-   }, {
-       #include "keymap_dv.h"
-   }, {
-       ... (fr, es, it, uk, us)
-   }
-};
-
```

```
-const char *CKeyMap::s_MapDirectory[] = { "DE","DV","ES","FR","IT","UK","US", 0 };
-
+// Zircon: the kernel compiles in NO country maps -- keyboard layouts ship as
+// SD:/etc/keymaps/*.kmap data files and are loaded at runtime through ClearTable/SetEntry
+// (see kapi_set_keymap_data). So a fresh keymap starts empty (every entry KeyNone).
 CKeyMap::CKeyMap (void)
 :  m_bCapsLock (FALSE), m_bNumLock (FALSE), m_bScrollLock (FALSE)
 {
-   const char *pLocale = CKernelOptions::Get ()->GetKeyMap ();
-   const void *pDefaultMap = LookupDefaultMap (pLocale);
-   ... (fallback to DEFAULT_KEYMAP)
-   memcpy (m_KeyMap, pDefaultMap, sizeof m_KeyMap);
+   memset (m_KeyMap, 0, sizeof m_KeyMap);      // KeyNone == 0
 }
@@ -336,17 +292,3 @@ u8 CKeyMap::GetLEDStatus (void) const
-const void *CKeyMap::LookupDefaultMap (const char *pLocale)
-{
-   for (unsigned nMap = 0; s_MapDirectory[nMap] != 0; nMap++)
-       if (strcmp (s_MapDirectory[nMap], pLocale) == 0)
-           return s_DefaultMap[nMap];
-   return 0;
-}
```

```
--- a/include/circle/input/keyboardbehaviour.h
+++ b/include/circle/input/keyboardbehaviour.h
@@ -69,6 +69,8 @@ public:
    u8 GetLEDStatus (void) const;

+   CKeyMap *GetKeyMap (void)   { return &m_KeyMap; }   // Zircon: runtime layout switch
+
 private:
    void GenerateKeyEvent (u8 ucKeyCode);
```

```
--- a/include/circle/usb/usbkeyboard.h
+++ b/include/circle/usb/usbkeyboard.h
@@ -62,6 +62,9 @@ public:
    boolean SetLEDs (u8 ucStatus);

+   // Zircon: access the cooked-mode key map so the layout can be switched at runtime.
+   CKeyMap *GetKeyMap (void)   { return m_Behaviour.GetKeyMap (); }
+
 private:
    void ReportHandler (const u8 *pReport, unsigned nReportSize);
```

```
--- a/include/circle/sysconfig.h
+++ b/include/circle/sysconfig.h
@@ -273,7 +277,8 @@
 #ifndef MAX_TASKS
-#define MAX_TASKS       20
+#define MAX_TASKS       40      // Onyx: bumped from 20 -- net stack adds
+                        // background tasks (net/DHCP/WPA + NTP/IRC)
 #endif
```

(The 7 `lib/input/keymap_*.h` tables are deleted — 905 lines — and not shown here.)

2. Heap large-block reuse (fix per-launch leak)

Why. Circle's `CHeapAllocator` keeps freed blocks on per-size **bucket** free lists, but the buckets top out at `0x80000` = 512 KB. A block **bigger than the top bucket cannot be returned to any free list and is lost** on `Free()`. Onyx allocates large blocks routinely — a window **canvas** is up to ~3 MB (1024×768×4) — so **every app launch leaked its canvas**.

Fix. Add **power-of-two "large" free lists** alongside the buckets: a too-big block is rounded to its power-of-two class and pushed onto `m_pLargeFreeList[exp]` on free, and reused from it on allocate. A running `m_nFreeListBytes` counter (exposed via `GetFreeListSpace()`) feeds the `meminfo` accounting (patch 3). **Rebuild `libcircle`** after this change.

```
--- a/include/circle/heapallocator.h
+++ b/include/circle/heapallocator.h
@@ -40,6 +40,12 @@ ASSERT_STATIC (DATA_CACHE_LINE_LENGTH_MAX >= 16);
 #define HEAP_BLOCK_MAX_BUCKETS 20

+// Onyx: blocks larger than the biggest bucket are rounded up to a power of two and
+// kept on a per-power free list (index = log2 of the rounded size), so large blocks
+// (window canvases, big file buffers, ...) are reused instead of lost on free. 32
+// classes cover any realistic size.
+#define HEAP_LARGE_LISTS   32
+
 struct THeapBlockHeader
@@ -83,6 +89,10 @@ public:
     size_t GetFreeSpace (void) const;

+   /// \return Onyx: bytes of freed blocks currently on the bucket/large free lists
+   ///     (reusable). Add to GetFreeSpace() for the true total free space.
+   size_t GetFreeListSpace (void) const    { return m_nFreeListBytes; }
+
     /// \param nSize Block size to be allocated
@@ -95,8 +105,8 @@ public:
     /// \param pBlock Memory block to be freed
-   /// \note Memory space of blocks, which are bigger than the largest bucket size,
-   ///   cannot be returned to a free list and is lost.
+   /// \note Onyx: blocks bigger than the largest bucket are rounded to a power of two
+   ///   and returned to a per-power free list (reused), so they are NOT lost.
     void Free (void *pBlock);
@@ -117,6 +127,8 @@ private:
     THeapBlockBucket m_Bucket[HEAP_BLOCK_MAX_BUCKETS+1];
+   THeapBlockHeader *m_pLargeFreeList[HEAP_LARGE_LISTS];   // per-power-of-2 (Onyx)
+   size_t       m_nFreeListBytes;  // Onyx: bytes on the bucket+large free lists
     CSpinLock    m_SpinLock;

--- a/lib/heapallocator.cpp
+++ b/lib/heapallocator.cpp
@@ -53,6 +53,45 @@ void CHeapAllocator::Setup (uintptr nBase, size_t nSize, size_t nReserve)
     m_nReserve = nReserve;
+
+   for (unsigned i = 0; i < HEAP_LARGE_LISTS; i++)     // Onyx: large-block free lists
+       m_pLargeFreeList[i] = 0;
+   m_nFreeListBytes = 0;
+}
+
+// Onyx: round a too-big request up to a power-of-two class; return its free-list index.
+static int HeapLargeClass (size_t *pnSize)
+{
```

```
+   size_t nRounded = HEAP_BLOCK_ALIGN; int nExp = 0;
+   while (nRounded < *pnSize) { nRounded <<= 1; if (++nExp >= HEAP_LARGE_LISTS) return -1; }
+   *pnSize = nRounded; return nExp;
+}
+// Free-list index of an already-rounded large block (stored size = ALIGN << exp).
+static int HeapLargeIndex (size_t nRoundedSize)
+{
+   int nExp = 0;
+   while (((size_t) HEAP_BLOCK_ALIGN << nExp) < nRoundedSize)
+       if (++nExp >= HEAP_LARGE_LISTS) return -1;
+   return nExp;
 }
@@ -96,12 +135,28 @@ void *CHeapAllocator::DoAllocate (size_t nSize)
+   // Onyx: too big for every bucket -> round to a power-of-two class so it can be
+   // reused from m_pLargeFreeList instead of being lost on free.
+   int nLargeExp = -1;
+   if (pBucket->nSize == 0) nLargeExp = HeapLargeClass (&nSize);
+
    THeapBlockHeader *pBlockHeader;
    if (pBucket->nSize > 0 && (pBlockHeader = pBucket->pFreeList) != 0) {
        pBucket->pFreeList = pBlockHeader->pNext;
+       m_nFreeListBytes -= pBlockHeader->nSize;    // Onyx
+   } else if (nLargeExp >= 0 && (pBlockHeader = m_pLargeFreeList[nLargeExp]) != 0) {
+       m_pLargeFreeList[nLargeExp] = pBlockHeader->pNext;
+       m_nFreeListBytes -= pBlockHeader->nSize;    // Onyx
    } else { ... bump m_pNext ... }
@@ -226,6 +281,7 @@ void CHeapAllocator::DoFree (void *pBlock)   // (bucket free path)
            pBucket->pFreeList = pBlockHeader;
+           m_nFreeListBytes += pBlockHeader->nSize;    // Onyx
@@ -237,6 +293,19 @@ void CHeapAllocator::DoFree (void *pBlock)
+   // Onyx: no matching bucket -> power-of-two "large" block: return it to its size
+   // class free list so it is reused on the next same-class allocation (no leak).
+   int nLargeExp = HeapLargeIndex (pBlockHeader->nSize);
+   if (nLargeExp >= 0) {
+       m_SpinLock.Acquire ();
+       pBlockHeader->pNext = m_pLargeFreeList[nLargeExp];
+       m_pLargeFreeList[nLargeExp] = pBlockHeader;
+       m_nFreeListBytes += pBlockHeader->nSize;
+       m_SpinLock.Release ();
+       return;
+   }

--- a/include/circle/sysconfig.h
+++ b/include/circle/sysconfig.h
@@ -85,6 +85,10 @@
 #ifndef HEAP_BLOCK_BUCKET_SIZES
+// Small fixed buckets for common sizes. Larger requests (e.g. window canvases) no
+// longer need a matching bucket: Onyx's heap allocator rounds anything above the top
+// bucket up to a power-of-two class and reuses it from a per-power free list (see
+// HEAP_LARGE_LISTS / heapallocator.cpp), so they are no longer lost on free.
 #define HEAP_BLOCK_BUCKET_SIZES    0x40,0x400,0x1000,0x4000,0x10000,0x40000,0x80000
 #endif
```

3. Free-space accounting (for the `meminfo` kapi)

Why. The `meminfo` kapi (and the memory monitor) reports total / free / app memory. The "free" figure must include both the unallocated region *and* the freed blocks/pages sitting on the allocators' free lists (reusable). Circle exposed only the unallocated region, so we add free-list accessors. `memory.h` is header-only; the `pageallocator` counter needs a `libcircle` rebuild.

```
--- a/include/circle/memory.h
+++ b/include/circle/memory.h
@@ -155,6 +155,25 @@ public:
     static void PageFree (void *pPage)  { s_pThis->m_Pager.Free (pPage); }

+   // Onyx: free space (bytes) of the page allocator region not yet handed out.
+   static size_t GetPagerFreeSpace (void)  { return s_pThis->m_Pager.GetFreeSpace (); }
+   // Onyx: + freed pages on the pager free list (reusable).
+   static size_t GetPagerFreeListSpace (void) { return s_pThis->m_Pager.GetFreeListSpace ();
 }
+
+   // Onyx: freed heap blocks on the bucket/large free lists (reusable). Add to
+   // GetHeapFreeSpace(HEAP_ANY) for the true free heap.
+   size_t GetHeapFreeListSpace (void) const
+   {
+#if RASPPI >= 4
+       return s_pThis->m_HeapLow.GetFreeListSpace () + s_pThis->m_HeapHigh.GetFreeListSpace
();
+#else
+       return s_pThis->m_HeapLow.GetFreeListSpace ();
+#endif
+   }
+
     static void DumpStatus (void)

--- a/include/circle/pageallocator.h
+++ b/include/circle/pageallocator.h
@@ -48,6 +48,10 @@ public:
     size_t GetFreeSpace (void) const;

+   /// \return Onyx: bytes of freed pages currently on the free list (reusable). Add
+   ///     to GetFreeSpace() for the true total free space.
+   size_t GetFreeListSpace (void) const;
+
     void *Allocate (void);
@@ -67,6 +71,7 @@ private:
     TFreePage   *m_pFreeList;
+   unsigned     m_nFreeListCount;  // Onyx: pages currently on m_pFreeList
     CSpinLock    m_SpinLock;

--- a/lib/pageallocator.cpp
+++ b/lib/pageallocator.cpp
@@ -30,7 +30,8 @@ CPageAllocator::CPageAllocator (void)
-   m_pFreeList (0)
+   m_pFreeList (0),
+   m_nFreeListCount (0)
 {
 }
@@ -49,6 +50,11 @@ size_t CPageAllocator::GetFreeSpace (void) const
+size_t CPageAllocator::GetFreeListSpace (void) const   // Onyx: reusable freed pages
+{
+   return (size_t) m_nFreeListCount * PAGE_SIZE;
+}
@@ -68,6 +74,7 @@ void *CPageAllocator::Allocate (void)
         m_pFreeList = pFreePage->pNext;
+       m_nFreeListCount--;         // Onyx: page taken off the free list
@@ -103,6 +110,7 @@ void CPageAllocator::Free (void *pPage)
     m_pFreeList = pFreePage;
+   m_nFreeListCount++;             // Onyx: page returned to the free list
```

Not a patch: build configuration

One required setting is a **configure option**, not a source change: Onyx renders 32-bit pixels, so Circle must be configured with `DEPTH=32`:

```
cd circle && ./configure -r 4 -p aarch64-none-elf- -d DEPTH=32 -f
```

See the Developer Guide §2 for the full Circle build.

Updating the fork (submodule)

```
# rebase the Onyx patches onto a newer upstream, inside the submodule:
git -C circle fetch upstream
git -C circle rebase upstream/master onyx     # resolve conflicts in the patched files
# rebuild the affected libraries: libcircle (heap + page accounting), libinput (keymap)
# then record the new fork commit in the superproject:
git add circle && git commit -m "Bump circle submodule"
```